АНАЛИТИЧЕСКИЙ ОТЧЁТ · ПРОКАЧКА НА ОТКРЫТОМ КОДЕ

Монтажная Кухня

Двенадцать простых апгрейдов карманного AI-видеостудио для друзей и блогеров — от аппаратного кодирования рендера и кадров-превью в карточках хуков до B-roll из Pexels и автоматического дубляжа. Всё легально и бесплатно: MIT, Public Domain, CC0.

Простые апгрейды в том же направлении

OpenShorts · supoclip · auto-editor · WhisperX · PySceneDetect · Pexels · freesound · FFmpeg VAAPI · Bot API 9.0

ОКТЯБРЬ 2026 · ВЕРСИЯ 1.0 · ВНУТРЕННИЙ ДОКУМЕНТ

Содержание

1. Резюме для владельца 2

1.1. Сводная таблица идей . . . 2

2. Текущее состояние проекта 3

3. Разведка открытых источников 2026 4

4. Gap-анализ: где мы сейчас 6

5. P0 — быстрые победы 7

5.1. Аппаратное кодирование h264_vaapi . . . 7

5.2. Кадры-превью в карточках хуков . . . 7

5.3. Ссылки со всех площадок . . . 8

6. P1 — плотность и звук 9

6.1. Реальная вырезка пауз внутри реплик . . . 9

6.2. Звуковые акценты под CC0 . . . 9

6.3. Превью за 10 секунд до рендера . . . 9

6.4. Границы сцен для нарезки . . . 10

7. P2 — качество картинки и интерфейса 10

7.1. B-roll из Pexels . . . 10

7.2. Музыка с авто-дакингом . . . 10

7.3. WhisperX: синхрон караоке и диаризация . . . 11

7.4. DeviceStorage из Bot API 9.0 . . . 11

7.5. Пять заголовков и две обложки . . . 11

8. P3 — авто-дубляж UA–RU 12

9. Дорожная карта, бюджет и анти-идеи 12

9.1. Что переносить не стоит . . . 13

1. Резюме для владельца

Настоящий отчёт — ответ на вопрос владельца: как простым способом прокачать «Монтажную Кухню», не меняя её направления. Мы повторно проанализировали состояние проекта после закрытия технического задания от 25 октября 2026 года — авто-полноэкранный режим, переключатель RU/UA, онбординг «три шага» и поток «ИИ-хуки» работают и заархивированы в продакшн-сборку. Затем мы прошлись по открытым источникам 2026 года: флагманским self-hosted генераторам клипов OpenShorts и supoclip, утилите auto-editor, набору WhisperX, детектору сцен PySceneDetect, бесплатным API Pexels и freesound, аппаратному кодированию FFmpeg и свежим возможностям Bot API 9.0. Отчёт отвечает на один практический вопрос: что из этого встраивается в наш конвейер за дни, а не недели.

Ключевой вывод утешительный: архитектурно проект уже зрелый, и никто из открытых конкурентов не предлагает того, что у нас есть бесплатно, — доставку готового ролика прямо в Telegram-чат, двуязычный интерфейс RU/UA, приоритетную очередь для задач друзей и нормализацию звука по EBU R128. Поэтому стратегия не «переписать», а «точечно усилить»: двенадцать идей разбиты на четыре приоритета, из которых три быстрые победы встраиваются за один-два дня каждая и не требуют ни новых сервисов, ни изменений в базе данных, ни бюджета. Все заимствования легальны: лицензии MIT, Public Domain, BSD и CC0, а также бесплатные квоты публичных API.

3 × 1–2 дня
быстрые победы P0 — без правок архитектуры и базы данных

3–5×
ускорение кодирования через h264_vaapi на Intel-графике ноутбука

0 руб.
бюджет всех заимствований: MIT, Public Domain, CC0, бесплатные API

1.1. Сводная таблица идей

Идея	Эффект для друга-блогера	Трудозатраты	Приоритет
Аппаратное кодирование VAAPI	Очередь движется в разы быстрее — ролик приезжает раньше	1–2 дня	P0
Кадры-превью в карточках хуков	Фрагмент выбирается глазами, а не по цифре балла	1 день	P0
Приём ссылок любых платформ	TikTok, Reels, Vimeo, Twitch — без перекачки вручную	полдня	P0
Реальная вырезка пауз в репликах	Плотный темп речи без «э-э» и зависаний	3–4 дня	P1
Звуки переходов под CC0	Динамика монтажа как у платных инструментов	2 дня	P1

Идея	Эффект для друга-блогера	Трудозатраты	Приоритет
Превью 10 секунд до рендера	Сомнительная нарезка не сжигает слот очереди	2 дня	P1
B-roll из Pexels и музыка с дакингом	«Картинка» уровня студийных клипов	1 неделя	P2
Дубляж UA–RU голосами edge-tts	Один ролик — две языковые аудитории	1–2 недели	P3

Таблица 1. Двенадцать идей апгрейда: восемь ключевых в порядке эффекта

Порядок чтения: главы 5–7 содержат рецепты с командами и точками интеграции в существующие стадии конвейера, глава 8 — флагманскую идею следующего горизонта, глава 9 — календарь на четыре недели, бюджет оперативной памяти и список того, что переносить не стоит. Дорожная карта построена так, чтобы каждую неделю заканчивать видимым улучшением для друзей и блогеров, а не внутренним рефакторингом: первая неделя — скорость и выбор глазами, вторая — плотность монтажа и звук, третья и четвёртая — картинка, охват и память стиля.

2. Текущее состояние проекта

Напомним, что уже работает. «Монтажная Кухня» — это Telegram-бот с Mini App и тяжёлым FIFO-воркером на ноутбуке с Linux Mint (12 ГБ оперативной памяти, FFmpeg 7.1, Faster-Whisper). Публичный контур закрывает полный цикл: друг присылает ссылку или видео, конвейер скачивает исходник, распознаёт речь, находит сильные моменты окнами 35–55 секунд, рендерит вертикальный ролик 1080×1920 с титрами, плашками и субтитрами (украинскими, русскими или двуязычными), нормализует громкость по EBU R128 и возвращает готовый MP4 прямо в чат. К ролику прикладывается пакет YouTube: название, описание с таймкодами и хэштеги — блогеру остаётся вставить его при публикации.

Интерфейс после доработки от 25 октября выглядит как взрослый продукт: приложение раскрывается на весь экран с кнопкой переключения режима окна, язык переключается между русским и украинским мгновенно и сохраняется между сессиями, вкладка «ИИ-хуки» встречает онбордингом из трёх шагов и тремя пресетами «в один клик», а технический жаргон заменён ценностными формулировками. Отдельно от публичного контура живут приватные механики: конвейер новостей Domdara с озвучкой edge-tts и модерацией, приватная выгрузка на YouTube через OAuth и админ-панель. Изоляция чистая: друзья не видят ничего приватного, монетизации в публичном боте нет и не планируется.

Возможность	Статус на 25.10.2026	Где живёт
Поиск хуков: окна 35–55 с, скоринг 60–99	Работает	services/worker/app/hook_scorer.py
Транскрипция с таймкодами слов	Работает	pipeline.py, Faster-Whisper
Субтитры укр/рус/двуязычные	Работает	render_short, формат ASS
Караоке-подсветка слова золотом	Работает, но только в новостях	tts_narrator.py — кандидат на перенос в клипы
Нормализация звука EBU R128	Работает	loudnorm I=-14:TP=-1.5:LRA=11
Приоритетная очередь: друзья P10, фон P1	Работает	claim_next, SQLite на BM
RU/UA, полноэкран, онбординг, пресеты	Работает	Mini App, i18n.tsx, onboarding-steps.tsx
Приём ссылок на исходник	Только YouTube	hooks-tab.tsx, YT_RE — ограничение
Кодирование видео	Процессор, libx264	engine.py, ENGINE_COMMANDS — узкое место

Таблица 2. Срез возможностей конвейера и две явные цели для апгрейда

Два ограничения из таблицы — ссылки только с YouTube и чисто процессорное кодирование — и становятся главными целями быстрых побед в главе 5: первое сужает круг источников друзей, второе удлиняет ожидание в очереди. Третье, менее очевидное, ограничение — карточки кандидатов показывают только текст и балл, хотя исходное видео к этому моменту уже лежит на диске ноутбука, и кадры из него стоят копейки. Отдельно отметим готовый фундамент: генератор караоке-субтитров уже написан в новостном конвейере, голоса edge-tts на двух языках уже прописаны — половина будущих фичей существует в коде и ждёт переиспользования.

3. Разведка открытых источников 2026

Открытый код по нашей тематике за год заметно повзрослел. Флагман среди self-hosted генераторов клипов — OpenShorts (лицензия MIT): он находит «вирусные моменты» по транскрипту и границам сцен, трекает лицо спикера связкой MediaPipe и YOLOv8 для умного кропа 9:16, поддерживает четыре режима кадрирования (трек лица, размытый фон, вертикальный сплит на двух спикерах с субтитрами на шве, скринкаст поверх спикера), делает дубляж на десятках языков и генерирует пакет YouTube из десяти вариантов названия с интерактивным выбором. Конкурент supoclip (лицензия AGPL-3.0, 1.7 тысячи звёзд, 427 коммитов на октябрь 2026) развивает редактор с шаблонами анимации субтитров и хук-заголовками, интерфейс на шести языках. Оба проекта активно коммитятся, но оба тянут тяжёлые зависимости: облачный Gemini для анализа и ElevenLabs для дубляжа.

Вторая группа — «точечные» инструменты, каждый из которых делает одно дело и встраивается за считанные дни. auto-editor (Public Domain, 5.6 тысяч звёзд, версия 31.7.2 от октября 2026) вырезает тишину и мёртвые паузы. WhisperX ускоряет распознавание до 70-кратного реального времени и добавляет выравнивание таймкодов по словам и диаризацию «кто сказал». PySceneDetect находит смены сцены по содержимому кадра и экспортирует таймкоды в формате, готовом для FFmpeg. Pexels открывает бесплатный API стоковых видео — 15 миллиардов запросов в месяц, фильтр портретной ориентации, поиск на 28 языках, требуется лишь атрибуция. freesound — архив звуковых эффектов под CC0, от свистов до импактов. Наконец, сам FFmpeg умеет аппаратное кодирование через VAAPI и QSV на Intel-графике, а Telegram Bot API 9.0 дал Mini App постоянное хранилище DeviceStorage.

Проект	Лицензия	Зрелость	Что берём в конвейер
OpenShorts	MIT	активен, полный цикл	Приёмы: четыре режима кадрирования, десять вариантов названия, превью до экспорта
supoclip	AGPL-3.0	1.7k звёзд, окт. 2026	Паттерн «хук-карточка с анимированной подписью»; аккуратно — только идеи, код под AGPL
auto-editor	Public Domain	5.6k звёзд, 157 релизов	Алгоритм: вырезка фрагментов тише −38 дБ длиннее 450 мс
WhisperX	BSD-2	v3.8.6, 2026	Выравнивание таймкодов слов для караоке-синхрона; диаризация для подкастов
PySceneDetect	BSD	эталон детекции	Таймкоды смен планов как границы нарезки, формат для FFmpeg
Pexels API	бесплатно, атрибуция	15 млрд запр./мес	Вертикальный B-roll под голос, фильтр portrait-ориентации
freesound	CC0 / Sampling+	архив университета	Пакет звуков переходов: свисты, импакты, whoosh
FFmpeg VAAPI	LGPL/GPL	встроен в 7.1	h264_vaapi — аппаратное кодирование на встроенной графике
Bot API 9.0	платформа	июль 2025	DeviceStorage — постоянное хранилище Mini App на устройстве

Таблица 3. Карта открытых источников: лицензии чистые, бюджет нулевой

Стратегия заимствования — «приёмы, а не код». Мы не тянем OpenShorts целиком: его конвейер завязан на облачный Gemini и собственную веб-витрину, а наш воркер обязан жить в 12 ГБ памяти и работать автономно на ноутбуке. Вместо этого мы переносим конкретные механики в существующие стадии: скоринг хуков у нас уже написан свой, караоке-генератор ASS уже написан для новостей — осталось применить его к клипам; аппаратное кодирование — это смена одной строки параметров FFmpeg. Единственная по-настоящему новая сущность — HTTP-клиент Pexels для B-roll, и это десяток строк в стадии подготовки ассетов. Отдельно подчеркнём лицензионную гигиену: код supoclip под AGPL мы в репозиторий не копируем, ограничившись паттернами интерфейса, а auto-editor и

PySceneDetect при желании вызываются как готовые CLI-инструменты без каких-либо обязательств.

4. Gap-анализ: где мы сейчас

Сравнение с открытыми флагманами честнее всего вести по строкам, где они сильны, и по строкам, где сильны мы. Главный наш козырь — дистрибуция: OpenShorts и supoclip отдают клипы в собственную веб-витрину, откуда их нужно скачивать руками, а наш ролик приезжает прямо в Telegram-чат, где его и ждёт блогер; ни один открытый аналог не делает этого из коробки. Второй козырь — цена: весь стек живёт на личном ноутбуке и микроскопической виртуалке за ноль рублей, тогда как полноценный self-hosted OpenShorts рассчитан на GPU-машину, а supoclip просит платные ключи Apify. Третий — двуязычные субтитры укр+рус в одной плашке и нормализация EBU R128 из коробки: уникальное сочетание для нашей аудитории.

Отстаём мы ровно в четырёх местах, и все четыре — «простые». Скорость: кодирование идёт на центральном процессоре, тогда как встроенная Intel-графика ноутбука простаивает. Наглядность: карточки кандидатов текстовые, хотя у лидеров выбранное место можно посмотреть глазами. Охват источников: ссылки принимаются только с YouTube, хотя yt-dlp, на котором построена стадия скачивания, поддерживает более тысячи площадок. Плотность: «Авто-вырезка» заявлена в интерфейсе, но внутри реплик паузы не вырезаются — конвейер режет по границам предложений, и «э-э» с зависаниями остаются в клипе.

Аспект	Монтажная Кухня сегодня	Лидеры открытого кода
Доставка результата	MP4 прямо в чат Telegram	Витрина в вебе, скачивание вручную
Стоимость владения	Ноутбук + микро-ВМ, 0 руб.	GPU-сервер или платные API-ключи
Субтитры и звук	Укр/рус/двуязычные, EBU R128	Анимированные шаблоны, дубляж 30+ языков
Выбор момента	Свой скоринг 60–99 + ручной контроль	LLM-скоринг, режим автопилота
Скорость рендера	Процессор libx264 — часы на длинном видео	Аппаратное кодирование — минуты
Наглядность выбора	Текст и балл	Кадры и клип-превью кандидата
Источники	Только YouTube	Более 1000 сайтов через yt-dlp
Паузы внутри реплик	Не вырезаются	auto-editor: тише −38 дБ — в нарезку

Таблица 4. Честный gap-анализ: четыре строки справа закрываются главами 5–6

«Наше преимущество — не количество фичей, а нулевое трение: друг присылает ссылку в чат и через полчаса получает готовый ролик туда же. Всё, что мы добавляем, обязано сохранять это трение нулевым.»

5. P0 — быстрые победы

Три победы этой главы объединяет одно: они не меняют ни архитектуру, ни базу данных, ни очередь — только параметры существующих стадий конвейера. Совокупная трудоёмкость — четыре-пять дней вместе с проверкой на живых видео. Начинать стоит именно с них: после них очередь друзей двигается быстрее, выбор хука — нагляднее, а круг источников — шире, то есть улучшается всё, что друзья замечают в первую очередь.

5.1. Аппаратное кодирование h264_vaapi

Ноутбук с Linux Mint почти наверняка несёт Intel-графику, а значит — аппаратный кодировщик H.264, доступный FFmpeg через VAAPI (открытый стек Intel: драйвер iHD, библиотека libva, пакет ставится из штатного репозитория). Кодирование — самая долгая чистая стадия нашего рендера: сегменты и финальная сборка прогоняются через libx264 на центральном процессоре, тогда как графика в это время простаивает. По замерам сообщества переход на аппаратный кодировщик даёт трёх-пятикратное ускорение кодирования, причём без роста потребления оперативной памяти — работа уходит в отдельный аппаратный блок. Для нашей FIFO-очереди это прямой выигрыш: слот рендера один, и любая его разгрузка ускоряет всех ожидающих.

Кодирование: libx264 → h264_vaapi, откат автоматически

```
# Проверка аппаратного кодировщика (Intel iGPU, драйвер iHD)
vainfo 2>/dev/null | grep -q VAProfileH264Enc && echo HW_OK

# Рендер сегмента: было -c:v libx264, стало аппаратное кодирование
ffmpeg -hwaccel vaapi -hwaccel_output_format vaapi -i seg.mp4 \
  -vf format=nv12|vaapi,hwupload -c:v h264_vaapi \
  -b:v 8M -maxrate 10M -bufsize 16M \
  -c:a aac -b:a 192k -movflags +faststart out.mp4
```

Интеграция живёт в engine.py, где сосредоточена единая точка правки ENGINE_COMMANDS: проверка vainfo при старте воркера задаёт флаг hw_encode, и команда финального рендера подставляет h264_vaapi вместо libx264; при любом сбое — откат на процессорный вариант, поэтому риск внедрения нулевой. Контроль качества обязателен: у VAAPI своя психовизуальная модель, поэтому первые клипы стоит сравнить глазами и убедиться, что битрейт 8 Мбит/с на 1080×1920 держит и лицо спикера, и мелкий текст плашек. Важно сохранить флаг faststart — ролик обязан стартовать в Telegram мгновенно, как сейчас.

5.2. Кадры-превью в карточках хуков

Сегодня карточка кандидата показывает балл, таймкод и цитату — этого мало для выбора «глазами», тем более что исходное видео к моменту анализа уже лежит на диске ноутбука. Решение почти бесплатное: после скоринга вытянуть по одному кадру из середины каждого окна, сохранить их в каталог задачи и отдать фронтенду через существующий API задач. Карточка получает фотографию момента — и блогер выбирает по выражению лица и композиции кадра, а не по цифре. У OpenShorts этот паттерн базовый: превью прямо в карточке кандидата, и именно он делает выбор уверенным.

Превью-кадры: секунды работы после стадии транскрипции

```
# Кадр-превью для карточки хука (start = граница кандидата)
ffmpeg -ss {start+5} -i source.mp4 -frames:v 1 \
  -vf scale=270:-2,setsar=1 thumbs/rank_{rank}.jpg -loglevel error
# Retina-вариант для чётких экранов: scale=540:-2
# Хранение: /data/media/{job_id}/thumbs, отдача через существующий API
```

Тонкость одна: кадры стоит тянуть из локального файла после стадии скачивания, а не во время анализа, чтобы не гонять лишний ввод-вывод; FFmpeg с -ss до -i делает это за долю секунды. Побочный бонус — те же кадры годятся как заготовки обложек: «кликбейтный стоп-кадр» уже выбирается слайдером в форме Shorts, и кадры кандидатов дают для него готовый материал без дополнительных прогонов. На фронте меняется только разметка карточки в hooks-tab.tsx: изображение, под ним — прежние балл и цитата.

5.3. Ссылки со всех площадок

Стадия скачивания уже построена на yt-dlp, который поддерживает более тысячи сайтов: TikTok, Instagram Reels, Vimeo, записи Twitch, X и другие. Ограничение живёт только на поверхности: регулярное выражение YT_RE во вкладке хуков, подсказка «только YouTube» в интерфейсе и симметричная проверка в боте. Расширение тривиально: любой http-ссылке разрешаем идти в анализ, название источника берём из метаданных yt-dlp, а подсказку меняем на «ссылка на видео». Для друзей, которые репостят друг у друга Reels и TikTok, это снимает главное трение: перекачивать видео файлом больше не нужно.

Две осторожности. Первая — TikTok и Reels приходят с водяным знаком площадки; честнее оставить его как есть и не обещать «чистую» картинку — это требование платформ к перепубликации. Вторая — некоторые площадки отдают поток со звуком в отдельных дорожках: yt-dlp склеивает их сам, но таймаут скачивания стоит поднять до щедрого. Оба случая закрываются в конфиге стадии скачивания: конвейер ниже не отличает источник — дальше работает единый путь «скачал — распознал — смонтировал».

6. P1 — плотность и звук

Этот уровень — про плотность монтажа и ощущение «дорогого» продакшена: паузы в речи, звуки переходов и возможность дёшево проверить нарезку до полного рендера. Каждая идея — от двух до четырёх дней, все в границах существующих стадий: правим список сегментов между транскрипцией и нарезкой, подмешиваем звук в финальной сборке, добавляем мини-задачу превью. Суммарный эффект воспринимается зрителем как «клип монтировал человек», хотя людей в конвейере по-прежнему ноль.

6.1. Реальная вырезка пауз внутри реплик

Формулировка «Авто-вырезка» уже красуется в интерфейсе как ценность — пора подкрепить её честной механикой. Сейчас конвейер режет по границам предложений из транскрипта, поэтому паузы внутри реплик и слова-связки остаются в клипе и разряжают темп. Рецепт auto-editor переносится на наш 16-килогерцовый wav, который и так генерируется на стадии извлечения аудио: детекция фрагментов тише −38 децибел и длиннее 450 миллисекунд, затем края соседних реплик прижимаются к границам паузы с сохранением 120 миллисекунд «дыхания», чтобы склейки не звучали вслепую. Список пауз достаточно смёржить со списком сегментов до нарезки — сама стадия рендера не меняется вовсе.

Вырезка пауз: детекция на готовом аудио, правка только таймкодов

```
# Паузы внутри реплик: на уже существующем 16k-wav (стадия аудио)
ffmpeg -i audio16k.wav -af silencedetect=noise=-38dB:d=0.45 \
  -f null - 2> silences.log   # парсим silence_start / silence_end

# Правка списка сегментов: края реплик прижать к границам паузы,
# сохранив 120 мс дыхания - клип "плотнеет" без склеек вслепую
```

6.2. Звуковые акценты под CC0

Второй по дешевизне приём после пауз — короткие звуки переходов: свист на смене плана, лёгкий импакт на появлении плашки-цитаты. На freesound такие звуки выложены под лицензией CC0 — полное общественное достояние, коммерческое использование разрешено без атрибуции; достаточно один раз собрать пакет из шести-восьми файлов и положить его рядом с ассетами воркера. Вставка — один amix в финальной сборке: громкость около −18 LUFS относительно речи, чтобы эффект «слышался», но не «выпирал». Платные конкуренты продают этот же приём как магию автоматизации; у нас он — константа в filtergraph и пять строк кода.

6.3. Превью за 10 секунд до рендера

Слот рендера — самый дефицитный ресурс системы, и тратить полчаса на нарезку, которая другу «не зашла», — расточительство. Лекарство — дешёвое превью: после выбора кандидата рендерим первые десять секунд нареза в 540×960 с черновыми субтитрами (секунды кодирования) и присылаем в чат с

кнопкой «Смонтировать целиком?». Подтверждение ставит задачу с обычным приоритетом друзей, отказ — мгновенно освобождает очередь. Паттерн «посмотри, потом плати рендером» подсмотрен у флагманов: у OpenShorts и supoclip превью показывается до экспорта, и именно это делает выбор уверенным, а очередь — занятой только нужной работой.

6.4. Границы сцен для нарезки

PySceneDetect умеет находить смены планов по содержимому кадра — детектор content-aware идёт по умолчанию — и экспортировать таймкоды в формате, который FFmpeg принимает напрямую. Для нас это способ сделать склейки «по картинке»: сейчас окно кандидата прижимается к границам предложений, но резать посреди плана — значит получить визуальный скачок на стыке. Правило простое: если в пределах полутора секунд от границы реплики есть смена сцены — сдвигаем край туда; иначе оставляем как есть. Анализ идёт по извлечённым кадрам быстрее реального времени и стоит копейки по памяти, поэтому встраивается в стадию анализа без риска для бюджета.

7. P2 — качество картинки и интерфейса

Второй горизонт — картинка и удобства: стоковый B-roll, музыка с автоматическим приглушением, точная синхронизация караоке, постоянное хранилище Telegram и вариативные заголовки с обложками. Здесь появляются первые «новые сущности» — HTTP-клиент Pexels и пакет музыки, — но и они встраиваются в существующие стадии без изменения схемы данных: всё это параметры задачи и файлы в каталоге ролика. Общая трудоёмкость блока — одна-две недели по выбору владельца.

7.1. B-roll из Pexels

Классический приём лидеров: в местах, где спикер говорит «в кадре нет действия», под голос подкладывается вертикальный стоковый ролик. Pexels API бесплатен, ищет по ключевым словам, фильтрует портретную ориентацию и отдаёт прямые ссылки на файлы; требуется лишь атрибуция — честно добавим строку «B-roll: Pexels» в YouTube-пакет. Ключевые слова берём из транскрипта: три самых частотных существительных сегмента через простой частотный анализ, без LLM. Вставка — та же пара scale и overlay, что уже используется для плашек и титров в render_short.

B-roll: один HTTP-запрос на сегмент, кеш на диск ноутбука

```
# B-roll: вертикальные клипы по ключевому слову (бесплатно, с атрибуцией)
GET https://api.pexels.com/videos/search?query=court&orientation=portrait
Authorization: <бесплатный ключ>
# Ответ: video_files[].link (выбираем hd_portrait, 9:16)
# Оверлей: overlay=...:enable='between(t,2,6)' поверх основного видео
```

7.2. Музыка с авто-дакингом

Фоновая музыка — самый быстрый способ придать клипу «продакшен». Пакет из десятка нейтральных треков с лицензией Creative Commons собирается один раз; в каждый ролик идёт один случайно выбранный. Ключ — не ручная громкость, а автоматический дакинг: фильтр sidechaincompress прижимает музыку под речь на 12–15 децибел, а в паузах возвращает обратно, ровно как делают живые звукорежиссёры. Эталон громкости остаётся EBU R128, поэтому музыка не ломает нормализацию — она подмешивается до loudnorm, и итоговый ролик проходит прежний контроль качества звука без изменений.

7.3. WhisperX: синхрон караоке и диаризация

Караоке-подсветка слов у нас уже реализована в новостном конвейере — генератор ASS с золотой подсветкой активного слова написан в tts_narrator.py. Перенос в клипы тривиален, но качество синхрона зависит от таймкодов слов: Faster-Whisper выдаёт их приближённо. WhisperX — надстройка над тем же Whisper с принудительным выравниванием слов и диаризацией спикеров: выравнивание делает подсветку миллисекундно-точной, а диаризация решает задачу подкастов с двумя голосами — плашки «Адвокат / Судья» или смена цвета субтитров по спикеру. Зависимость тяжёлая только на первый взгляд: она живёт в собственной стадии транскрипции и в пик рендера не попадает.

7.4. DeviceStorage из Bot API 9.0

Mini App хранит выбор языка и скрытие онбординга в localStorage — это переживает перезапуски, но не очистку данных браузера. Bot API 9.0 добавил постоянное хранилище DeviceStorage, которое живёт на устройстве пользователя под эгидой Telegram и не зависит от кеша веб-просмотра. Перенос тривиален: обёртка хранилища в модуле локализации получает второй бэкенд с откатом на localStorage для старых клиентов. Вместе с языком туда логично переложить последний выбранный пресет «в один клик» — тогда приложение запоминает «стиль» блогера между сессиями и устройствами.

7.5. Пять заголовков и две обложки

YouTube-пакет сегодня генерирует один вариант названия. Открытые флагманы делают десять вариантов с интерактивным выбором; нам достаточно пяти — по одному на каждый тип хука: вопрос, цифра, конфликт, интрига, прямое заявление. Половина механики уже есть: LLM-рефайнер скоринга умеет возвращать заголовки, формат сохранения — списком в параметрах задачи. Вторая половина — обложки: генератор изображений в проекте уже есть, и два варианта кликбейт-обложки (стоп-кадр с плашкой против полностью генеративной) дают блогеру выбор без похода в сторонние редакторы.

8. P3 — авто-дубляж UA–RU

Флагман следующего горизонта — автоматический дубляж между украинским и русским. «Двойной охват» сегодня означает двуязычные субтитры; следующий шаг — звук. Механика voice-over: транскрипт уже известен с точностью до слов, переводим его LLM-рефайнером (тот же вызов, что генерирует заголовки), синтезируем голосом edge-tts — украинская речь голосами Ostap или Polina, русская Dmitry или Svetlana, все четверо уже прописаны в нашем tts_narrator для новостей — и подмешиваем поверх оригинала, приглушённого до десяти процентов громкости. Синхронизация губ не требуется: для судеб, интервью и подкастов дубляж «за кадром» — принятая индустриальная норма.

Дакинг-дубляж: перевод + edge-tts + амикс в один проход

```
# Голос TTS поверх приглушённого оригинала (voice-over, не lipsync)
ffmpeg -i clip.mp4 -i dub_ru.mp3 -filter_complex \
  "[0:a]volume=0.10[bg];[bg][1:a]amix=inputs=2:duration=first:weights=1 4, \
  loudnorm=I=-14:TP=-1.5:LRA=11[a]" \
  -map 0:v -map "[a]" -c:v copy -c:a aac -b:a 192k out_ru.mp4
```

Ценность — мультипликатор аудитории без ручного труда: украинский контент становится доступным русскоязычной аудитории и наоборот, а блогеру не нужно делать ничего, кроме галочки «Двойной охват — звук». Риски честные. Перевод юридических формулировок требует аккуратного промпта — «не добавлять смысл, не смягчать формулировки». Тайминг русского и украинского предложений расходится на 10–15 процентов — лечится лёгким atempo в пределах 0.95–1.08 без потери естественности. Голоса edge-tts держим в LUFS-коридоре средствами уже работающего loudnorm. Именно поэтому идея стоит в P3, а не раньше: она требует настройки качества на живых роликах и осторожного выката сначала на одном типе контента.

> *«Дубляж — это не замена оригинала, а второй вход на то же видео: один исходник, две аудитории, ноль дополнительной работы для блогера.»*

9. Дорожная карта, бюджет и анти-идеи

Календарь построен по принципу «каждую неделю — видимое улучшение». Первая неделя закрывает все три быстрые победы: скорость кодирования, кадры в карточках и мультиплатформенные ссылки — это дни работ и немедленный отклик очереди. Вторая неделя делает монтаж плотным и звучным: паузы, звуки переходов и дешёвое превью. Третья и четвёртая недели — по выбору владельца: B-roll с музыкой для «картинки» либо WhisperX с заголовками для «мозгов» системы; дубляж стартует после них как отдельная задача на одну-две недели. Каждый пункт заканчивается проверкой на живом видео и решением «оставляем или откатываем» — половина идей открытого кода существует именно для того, чтобы её попробовали и выбрали лучшее.

Идея	Приоритет	Трудозатраты	Пик ОЗУ	Эффект
Аппаратное кодирование h264_vaapi	P0	1–2 дня	+0	Рендер в 3–5 раз быстрее
Кадры-превью хук-карточек	P0	1 день	+0.05 ГБ	Выбор глазами, рост конверсии «Смонтировать»
Ссылки всех платформ	P0	полдня	+0	TikTok, Reels, Vimeo, Twitch в одном чате
Вырезка пауз в репликах	P1	3–4 дня	+0	Плотный темп без слов-связок
Звуки переходов CC0	P1	2 дня	+0	Динамика, ощущение продакшена
Превью 10 секунд	P1	2 дня	+0	Экономия слота рендера на спорных нарезках
Границы сцен PySceneDetect	P1	3 дня	+0.1 ГБ	Склейки без визуальных скачков
B-roll из Pexels	P2	3–4 дня	+0.2 ГБ	Картинка уровня платных инструментов
Музыка с авто-дакингом	P2	2–3 дня	+0	Фон, не мешающий речи
WhisperX + перенос караоке	P2	4–5 дней	+1.5–2 ГБ	Миллисекундный синхрон подсветки слов
DeviceStorage (Bot API 9.0)	P2	1 день	+0	Память языка и пресета на устройстве
Дубляж UA–RU	P3	1–2 недели	+0.3 ГБ	Двойная аудитория одного ролика

Таблица 5. Мастер-план: приоритеты, трудозатраты и бюджет памяти

Бюджет оперативной памяти не меняется: почти все идеи либо исполняются внутри существующего процесса FFmpeg, либо живут в стадиях до пика рендера сегментов (сегодняшний максимум — 7.6 ГБ при запасе до лимита в 12 ГБ). Единственная зависимая от памяти — WhisperX: полная модель с диаризацией добавляет полтора-два гигабайта, поэтому она запускается в собственной стадии транскрипции, когда рендер ещё не начался; при нехватке — режим только выравнивания без диаризации. VAAPI не только не ест память — он ещё и освобождает процессор для параллельных стадий конвейера, поэтому его место в первой неделе не обсуждается.

9.1. Что переносить не стоит

Развилка «что не переносить» не менее важна, чем список заимствований, и здесь четыре твёрдых «нет». Первое — UGC-актёры и полная генерация «говорящих голов» из OpenShorts: это территория синтетического контента, а наш вектор — обработка настоящих записей друзей. Второе — автопостинг в TikTok и Reels: официальных API у площадок нет, неофициальные приводят к банам аккаунтов, и ручная публикация с готовым YouTube-пакетом остаётся честной границей автоматизации. Третье — монетизация в любом виде: Stars, подписки, тарифы прямо исключены владельцем, публичный бот остаётся подарком для друзей. Четвёртое — клонирование OpenShorts

целиком: тяжёлый стек, привязка к облачному Gemini и собственная витрина нам не нужны; наши сильные стороны — лёгкость, автономность и доставка в чат.

Если выбрать из всего отчёта одно действие — это первая неделя: три победы P0 дают максимально ощутимый эффект на несколько дней работы и не несут рисков. Дальнейший порядок гибкий: плотность монтажа важнее картинки для судебных клипов, картинка важнее для блогерских, дубляж подождёт обеих. Метрики успеха просты и уже измеримы нашими средствами: время от ссылки до ролика в чате, доля кандидатов, смонтированных с первого выбора, и количество платформ в истории задач — все три числа после месяца апгрейдов должны двигаться в правильную сторону.